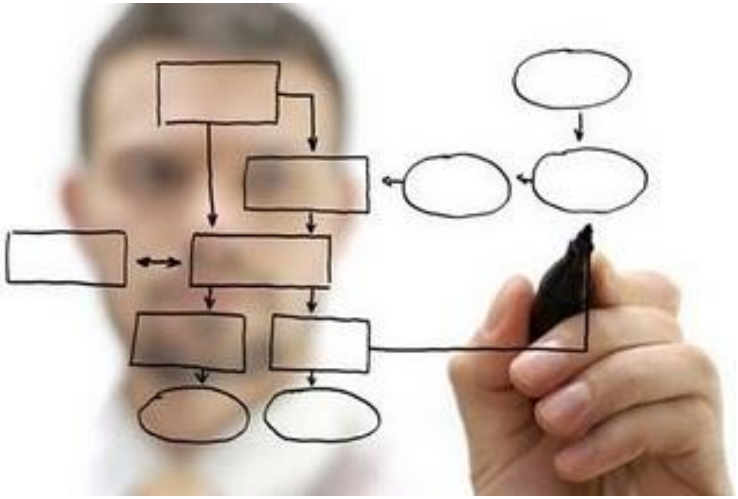


LINGO

DEIOAC-UPV

# LENGUAJES DE MODELIZACIÓN

## FORMULACIÓN DE MODELOS DE PROGRAMACIÓN MATEMÁTICA DE GRAN ESCALA:

Los modelos de programación matemática pueden ser de diferentes tamaños. Aunque algunos pueden incluir un pequeño conjunto de restricciones funcionales, en la práctica los modelos requieren cientos y hasta miles de restricciones. El número de variables decisión es incluso mayor que el de restricciones y ocasionalmente puede ser de cientos de miles. La formulación de modelos de tal dimensión resulta inabordable aun cuando los parámetros puedan tomarse desde hoja de cálculo. Es imprescindible utilizar un **lenguaje de modelización**.

# 4 Lenguajes de Modelización

- Los lenguajes de modelización son sistemas software específicamente diseñados para la formulación eficiente de modelos de programación matemática de gran tamaño. Se trata de lenguajes "*no procedurales*" sino lenguajes de especificación de modo que cuando introducimos un problema sólo podemos expresar lo que queremos, pero no cómo hacerlo, él se ocupa de cómo hacerlo. Con estos lenguajes expresamos nuestro problema de una forma sencilla, muy parecida a la escritura matemática habitual.
- Los modelos de gran tamaño aunque implican miles de restricciones funcionales, éstas son habitualmente de un número relativamente pequeño de tipos y las restricciones del mismo tipo siguen el mismo patrón. Por tanto utilizando largos bloques de datos desde hojas de cálculo o desde bases de datos, el lenguaje de modelización formulará todas las restricciones del mismo tipo trabajando simultáneamente con las variables de cada tipo.

# 4 Lenguajes de Modelización

Además de la formulación eficiente de modelos de gran tamaño, los lenguajes de modelización permiten agilizar otras tareas relacionadas con la modelización tales como:

- Acceso de datos
- Transformación de datos en parámetros del modelo
- Modificación del modelo cuando sea necesario
- Análisis de la solución
- Generar informes así como documentos sobre el contenido del modelo.

# 4 Lenguajes de Modelización

En las últimas décadas se han desarrollado varios lenguajes de modelización, entre los que se encuentran:

- AMPL
- MPL
- GAMS
- AIMMS
- PYOMO
- LINGO

# 4 Lenguajes de Modelización

- **LINGO** es un lenguaje de modelización diseñado particularmente para la formulación y resolución de una amplia variedad de problemas de optimización incluyendo programación lineal, programación entera y programación no lineal.
- **LINGO** tiene incorporada una librería con funciones matemáticas, estadísticas y financieras que nos permiten expresar fórmulas complejas de una manera clara y sencilla. Podemos representar nuestros datos dentro del programa en forma natural como una lista, pero LINGO también puede leer los datos desde ficheros externos o desde hojas de cálculo.

A continuación introduciremos los principios básicos del lenguaje de modelización LINGO. Utilizaremos para ello un ejemplo de planificación de la producción. En primer lugar formularemos el modelo algebraicamente según la sintaxis de LINGO y esta formulación nos servirá de apoyo para introducir los conceptos básicos de modelización con el lenguaje LINGO.

***El problema:***

La empresa PRODUCTSA desea determinar la combinación de productos a fabricar semanalmente. Cada unidad de producto requiere una determinada cantidad de tiempo de producción en tres máquinas. Cada máquina tiene un número de horas de producción disponibles por semana y cada producto tiene su beneficio asociado.

En la siguiente tabla se recogen los datos anteriormente mencionados.

| | TIEMPO DE PRODUCCIÓN POR UNIDAD (hrs) | | | | Disponibilidad semanal (hrs) |
|---|---|---|---|---|---|
| | PRODUCTO | | | | |
| MAQUINA | P1 | P2 | P3 | P4 | |
| Alisadora | 1.7 | 2.1 | 1.4 | 2.4 | 28 |
| Cortadora | 1.1 | 2.5 | 1.7 | 2.6 | 34 |
| Soldadora | 1.6 | 1.3 | 1.6 | 0.8 | 21 |
| Beneficio unitario | 26 | 35 | 25 | 37 | |

El objetivo a considerar es determinar cuál es la cantidad a producir de cada producto de modo que se maximice el beneficio sin sobrepasar la capacidad de producción de cada máquina.

# 4 Lenguajes de Modelización

**Algebraicamente** el modelo anterior queda formulado de la siguiente forma:

*Sea: xj = unidades fabricadas del producto Pj*

*cj = beneficio por unidad del producto Pj*

*aij = tiempo de producción en la máquina i por unidad del producto Pj*

*bi = tiempo de producción disponible por semana en la máquina i*

La función objetivo a considerar será:

$$\textit{Maximize} \quad \sum_{j=1}^{4} c_j x_j$$

y las restricciones: $\sum_{j=1}^{4} a_{ij} x_j \leq b_j$ *for i=1, 2, 3;*

# 4 Lenguajes de Modelización

Como ya se ha comentado, la potencia de los lenguajes de modelización se encuentra en que en un número reducido de líneas se pueden expresar una gran cantidad de restricciones. La clave de esta propiedad se encuentra en el concepto de **conjunto**.

Los **conjuntos** (***SETS***) son simplemente grupos de objetos relacionados. Un conjunto puede ser, por ejemplo, una lista de productos, tareas o almacenes. Cada elemento del conjunto puede tener una o más características asociadas a él; las llamamos **atributos**. Estos atributos pueden ser datos o pueden ser variables. LINGO reconoce dos tipos de conjuntos:

- **Conjunto primitivo** es el que está compuesto por objetos que no pueden reducirse más. *Por ejemplo,* un conjunto de 5 almacenes o un conjunto de 200 alumnos.
- **Conjunto derivado** es el que se puede construir a partir de conjuntos primitivos. *Por ejemplo*, el conjunto formado por las rutas entre cinco fábricas y diez ciudades es un conjunto derivado de los conjuntos de *fábricas* y *ciudades* que son dos conjuntos primitivos. También pueden construirse conjuntos derivados usando otros conjuntos derivados.

# 4 Lenguajes de Modelización

En general, un programa LINGO está formado por tres partes o secciones:

1. Sección **SETS**, que especifica los sets o conjuntos de objetos del modelo y sus atributos. Puede entenderse como la descripción de las estructuras de datos del problema.
2. Sección **DATA** que incluye tanto los datos a utilizar como la forma de obtenerlos.
3. Una sección que incluye el propio **modelo matemático**.

Opcionalmente, antes del modelo matemático, se puede incluir la sección **INIT**. Esta sección se utiliza para proporcionar valores iniciales a las variables del modelo antes de que comience la optimización para mejorar la eficiencia de la resolución del modelo.

# 4 Lenguajes de Modelización

**La sección SETS** se utiliza para definir conjuntos. Esta sección empieza con **SETS:** y acaba con **ENDSETS**. Entre estas dos palabras se definen los conjuntos que se van a utilizar.

Definimos un conjunto mediante su nombre, sus miembros y sus atributos. La sintaxis es:

**SETS:**

**nombre: atributos ;**

**ENDSETS**

```
SETS:
Maquina /alisadora, cortadora, soldadora/ : HrsDisponibles;
Producto /P1, P2, P3, P4/ : Beneficio, UProducidas;
ENDSETS
```

Los miembros de un SET se pueden enumerar separados por comas (“,”), o utilizando rangos (por ejemplo 1..8). Los atributos (características que nos interesan de estos conjuntos), si los hubiera, también se separan mediante comas. La definición de un set acaba en “;”

# 4 Lenguajes de Modelización

Otra información clave en el ejemplo es el número de horas de producción que cada unidad de producto utiliza en cada una de las máquinas. Este número puede considerarse con un atributo para los miembros del SET de todas las combinaciones de máquina y producto. Dado que este *SET* se deriva de los dos sets anteriores, se hace referencia al mismo como un **SET *derivado***. Se definirá del siguiente modo:

**MaPr(Maquina, Producto): HrsProdUtilizadas;**

- Las variables asociadas a conjuntos, es decir, los atributos, deben declararse antes de ser usadas.
- Los conjuntos derivados deben declararse después de los correspondientes conjuntos primitivos.

# 4 Lenguajes de Modelización

**La sección DATA** nos sirve para introducir valores a miembros de los sets y a los atributos. Esta sección empieza con **DATA:** y acaba con **ENDDATA**. La sintaxis es:

**DATA:**

**Set = lista de valores;**

**atributo = lista de valores ;**

**ENDDATA**

```
DATA:
HrsDisponibles = 28 34 21;
Beneficio =26 35 25 37;
HrsProdUtilizadas=1.7 2.1 1.4 2.4 !alisadora;
                  1.1 2.5 1.7 2.6 !cortadora;
                  1.6 1.3 1.6 0.8; !soldadora;
ENDDATA
```

Los valores se separan por comas o por espacios en blanco. Es preferible separarlos por comas porque cuando no queramos introducir un valor podemos dejar un hueco en blanco entre comas. Si queremos no explicitar un dato hasta el momento de ejecutar el programa podemos poner el signo "?" en su lugar; en ese momento LINGO nos preguntará por el correspondiente valor.

# 4 Lenguajes de Modelización

Para **especificar el modelo matemático** asociado al problema se utilizan **funciones** que permiten aplicar operaciones a los miembros de un set:

function(**set** | condition: **expresion**);

La parte "| condición" es optativa y sirve para restringir el "**set**".

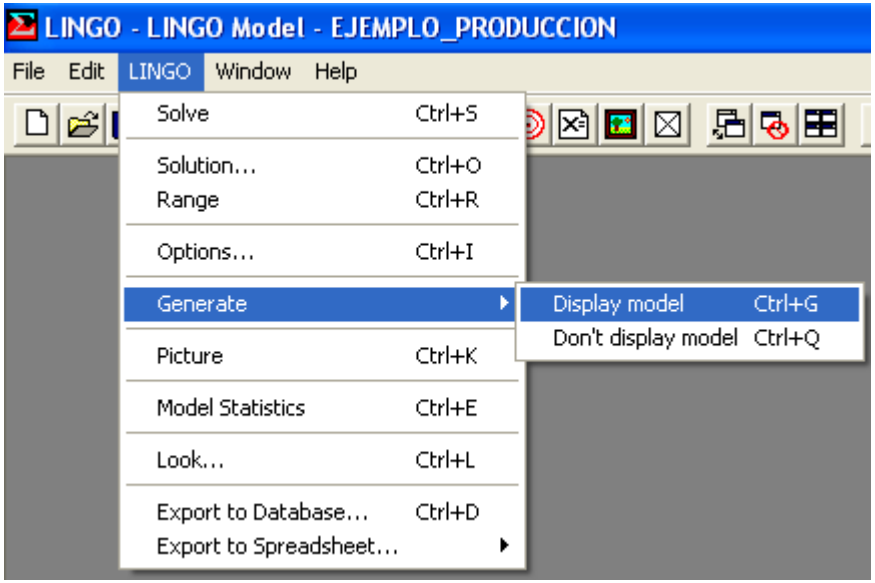
**@SUM(set: expresion)** Devuelve la suma de la expresión especificada sobre **set**.

**@SUM (Producto(j): Beneficio(j)* UProducidas(j))**

Esta función suma la expresión que aparece a continuación del símbolo "**:**" sobre todos los miembros del set que aparece antes de "**:**". Por tanto, esta función @SUM proporcionaría el valor correspondiente a la función objetivo.

# 4 Lenguajes de Modelización

**@SUM (Producto(j): Beneficio(j)* UProducidas(j))**




**26 UPRODUCIDAS( P1) +**
**35 UPRODUCIDAS( P2) +**
**25 UPRODUCIDAS( P3) +**
**37 UPRODUCIDAS( P4)**

# 4 Lenguajes de Modelización

**@FOR(set : constraint)** Genera restricciones independientemente para cada elemento del conjunto **set.**

```
@FOR(Maquina(i):
[Capacidad]@SUM(Producto(j):HrsProdUtilizadas(i,j)*UProducidas(j))
                                        <= HrsDisponibles(i););
```

```
CAPACIDAD( ALISADORA)]  1.7 UPRODUCIDAS( P1) + 2.1 UPRODUCIDAS( P2)
    + 1.4 UPRODUCIDAS( P3) + 2.4 UPRODUCIDAS( P4) <=   28

CAPACIDAD( CORTADORA)]  1.1 UPRODUCIDAS( P1) + 2.5 UPRODUCIDAS( P2)
    + 1.7 UPRODUCIDAS( P3) + 2.6 UPRODUCIDAS( P4) <=   34

CAPACIDAD( SOLDADORA)]  1.6 UPRODUCIDAS( P1) + 1.3 UPRODUCIDAS( P2)
    + 1.6 UPRODUCIDAS( P3) + .8 UPRODUCIDAS( P4) <=   21
```

# 4 Lenguajes de Modelización

Utilizando el lenguaje de especificación **LINGO**, el modelo resultante es el siguiente:

```
!Ejemplo de Planificación de la producción;

SETS:
        Maquina / alisadora, cortadora, soldadora /: HrsDisponibles;
        Producto / P1, P2, P3, P4/: Beneficio, UProducidas;
        MaPr(Maquina, Producto): HrsProdUtilizadas;
ENDSETS
DATA:
        HrsDisponibles = 28 34 21;
        Beneficio =26 35 25 37;
        HrsProdUtilizadas=      1.7 2.1 1.4 2.4 !alisadora;
                                2.5 1.7 2.6 !cortadora;
                                1.3 1.6 0.8; !soldadora;
ENDDATA
```

# 4 Lenguajes de Modelización

Utilizando el lenguaje de especificación **LINGO**, el modelo resultante es el siguiente:

```
!Maximizar el beneficio total;
MAX=@SUM(Producto(j):Beneficio(j)*UProducidas(j));


!Para cada máquina, las horas utilizadas deben ser <= horas disponibles;
@FOR(Maquina(i):
 [Capacidad]@SUM(Producto(j):HrsProdUtilizadas(i,j)*UProducidas(j))
                                <= HrsDisponibles(i););
```

# 4 Lenguajes de Modelización

LINGO - LINGO Model - EJEMPLO_PRODUCCION2

File Edit LINGO Window Help

LINGO Model - EJEMPLO_PRODUCCION2

```
!Ejemplo de Planificación de la producción;
!Definición de Sets primitivos;
SETS:
Maquina /alisadora, cortadora, soldadora/: HrsDisponibles;
Producto /P1, P2, P3, P4/: Beneficio, UProducidas;
!Definición de Sets Derivados;
MaPr(Maquina, Producto): HrsProdUtilizadas;
ENDSETS
DATA:
!Horas disponibles de cada máquina;
HrsDisponibles = 28 34 21;
!Beneficio unitario;
Beneficio =26 35 25 37;
!Horas de máquina necesarias por unidad de producto;
HrsProdUtilizadas= 1.7 2.1 1.4 2.4 !alisadora;
                    1.1  2.5 1.7 2.6 !cortadora;
                    1.6  1.3 1.6 0.8; !soldadora;
ENDDATA
!Maximizar el beneficio total;
MAX=@SUM(Producto(j):Beneficio(j)*UProducidas(j));
!Para cada máquina, las horas deben ser <= horas disponibles;
@FOR(Maquina(i):
[Capacidad]@SUM(Producto(j):HrsProdUtilizadas(i,j)*UProducidas(j))
                    <= HrsDisponibles(i);
     );
```

# 4 Lenguajes de Modelización

```
MAX     26 UPRODUCIDAS( P1) + 35 UPRODUCIDAS( P2) + 25
  UPRODUCIDAS( P3) + 37 UPRODUCIDAS( P4)
SUBJECT TO
CAPACIDAD( ALISADORA)]  1.7 UPRODUCIDAS( P1) + 2.1
  UPRODUCIDAS( P2)
    + 1.4 UPRODUCIDAS( P3) + 2.4 UPRODUCIDAS( P4) <=   28
CAPACIDAD( CORTADORA)]  1.1 UPRODUCIDAS( P1) + 2.5
  UPRODUCIDAS( P2)
    + 1.7 UPRODUCIDAS( P3) + 2.6 UPRODUCIDAS( P4) <=   34
CAPACIDAD( SOLDADORA)]  1.6 UPRODUCIDAS( P1) + 1.3
  UPRODUCIDAS( P2)
    + 1.6 UPRODUCIDAS( P3) + .8 UPRODUCIDAS( P4) <=   21
END
```

# 4 Lenguajes de Modelización

Solution Report - EJEMPLO_PRODUCCION2

```
Global optimal solution found at step:              6
Objective value:                              475.0000

                     Variable           Value        Reduced Cost
    HRSDISPONIBLES( ALISADORA)        28.00000           0.0000000
    HRSDISPONIBLES( CORTADORA)        34.00000           0.0000000
    HRSDISPONIBLES( SOLDADORA)        21.00000           0.0000000
                BENEFICIO( P1)        26.00000           0.0000000
                BENEFICIO( P2)        35.00000           0.0000000
                BENEFICIO( P3)        25.00000           0.0000000
                BENEFICIO( P4)        37.00000           0.0000000
               UPRODUCIDAS( P1)       0.0000000           3.577921
               UPRODUCIDAS( P2)        10.00000           0.0000000
               UPRODUCIDAS( P3)        5.000000           0.0000000
               UPRODUCIDAS( P4)       0.0000000           1.441558
 HRSPRODUTILIZADAS( ALISADORA,        1.700000           0.0000000
 HRSPRODUTILIZADAS( ALISADORA,        2.100000           0.0000000
 HRSPRODUTILIZADAS( ALISADORA,        1.400000           0.0000000
 HRSPRODUTILIZADAS( ALISADORA,        2.400000           0.0000000
 HRSPRODUTILIZADAS( CORTADORA,        1.100000           0.0000000
 HRSPRODUTILIZADAS( CORTADORA,        2.500000           0.0000000
 HRSPRODUTILIZADAS( CORTADORA,        1.700000           0.0000000
 HRSPRODUTILIZADAS( CORTADORA,        2.600000           0.0000000
 HRSPRODUTILIZADAS( SOLDADORA,        1.600000           0.0000000
 HRSPRODUTILIZADAS( SOLDADORA,        1.300000           0.0000000
 HRSPRODUTILIZADAS( SOLDADORA,        1.600000           0.0000000
 HRSPRODUTILIZADAS( SOLDADORA,       0.8000000           0.0000000

                            Row    Slack or Surplus      Dual Price
                              1        475.0000           1.000000
         CAPACIDAD( ALISADORA)       0.0000000           15.25974
         CAPACIDAD( CORTADORA)       0.5000000           0.0000000
         CAPACIDAD( SOLDADORA)       0.0000000           2.272727
```

# 4 Lenguajes de Modelización

**OPERADORES LÓGICOS Y FUNCIONES MATEMÁTICAS EN LINGO:**

**#NOT#** : Es un operador unitario, niega el valor lógico de su argumento.

**#AND#** : Devuelve el valor TRUE sólo si sus dos argumentos son TRUE, en otro caso devuelve el valor FALSE.

**#OR#** : Devuelve el valor FALSE sólo si sus dos argumentos son FALSE, en otro caso devuelve el valor TRUE.

**#EQ#** : Devuelve TRUE si sus dos operandos son iguales y FALSE en caso contrario.

**#NE#** : Devuelve TRUE si sus dos operandos no son iguales y FALSE si lo son.

**#GT#** : Devuelve TRUE si el operando izquierdo es mayor que el derecho, y FALSE en caso contrario.

**#GE#** : Devuelve TRUE si el operando izquierdo es mayor o igual que el derecho, y FALSE en caso contrario.

**#LT#** : Devuelve TRUE si el operando izquierdo es menor que el derecho, y FALSE en caso contrario.

**#LE#** : Devuelve TRUE si el operando izquierdo es menor o igual que el derecho, y FALSE en caso contrario

# 4 Lenguajes de Modelización

Otras funciones en LINGO:

**@MAX (set: expresion);**

Devuelve el valor máximo de **expresion** tomado sobre **set**.

**@MIN (set: expresion);**

Devuelve el valor mínimo de **expresion** tomado sobre **set**.

**@ABS(X);** Devuelve el valor absoluto de X.

**@SMAX(lista);** Devuelve el máximo de un conjunto de valores.

**@SMIN(lista);** Devuelve el mínimo de un conjunto de valores.

## IMPORTACIÓN Y EXPORTACIÓN DE DATOS A HOJA DE CALCULO CON LINGO:

- El ejemplo que acabamos de ver es auto-contenido ya que todos los datos estaban incorporados en la formulación de LINGO. Sin embargo en la práctica es frecuente que los datos se encuentren almacenados en otra fuente (habitualmente Hoja de Cálculo) desde la cual necesitan incorporarse al modelo.
- Para responder a esta cuestión, LINGO dispone del comando **@OLE()**, para cargar o almacenar datos desde o a una hoja de cálculo.

HrsProdUtilizadas | fx 1,7

| | A | B | C | D | E | F | G | H | I | J |
|---|---|---|---|---|---|---|---|---|---|---|
| 1 | | | | | | | | | | |
| 2 | | | | | | | | | | |
| 3 | EJEMPLO PLANIFICACION DE LA PRODUCCION | | | | | | | | | |
| 4 | | | | | | | | | | |
| 5 | | | | | | | | | | |
| 6 | BENEFICIO | | | | P1 | P2 | P3 | P4 | | |
| 7 | | Beneficio/unidad: | | | 26 | 35 | 25 | 37 | | |
| 8 | | Unidades a Producir: | | | | | | | | |
| 9 | | | | | | | | | | |
| 10 | | | | | | | | | | |
| 11 | MAQUINA | | HrsDisponibles | | Hrs. Utilizadas | Hrs. Producción utilizadas por unidad de producto | | | | |
| 12 | ALISADORA | | 28 | >= | | 1,7 | 2,1 | 1,4 | 2,4 | |
| 13 | CORTADORA | | 34 | >= | | 1,2 | 2,5 | 1,7 | 2,6 | |
| 14 | SOLDADORA | | 21 | >= | | 1,7 | 1,3 | 1,6 | 0,8 | |
| 15 | | | | | | | | | | |

## IMPORTACIÓN Y EXPORTACIÓN DE DATOS A HOJA DE CALCULO CON LINGO:

Un paso previo para poder utilizar de forma efectiva la función **@OLE()** es la asignación de los **nombres apropiados** a los rangos de datos correspondientes

| P1 | P2 | P3 | P4 |
|---|---|---|---|
| 26 | 35 | 25 | 37 |

| Hrs. Utilizadas | Hrs. Producción utilizadas por unida | | |
|---|---|---|---|
| 0 | 1,7 | 2,1 | 1,4 |
| 0 | 1,2 | 2,5 | 1,7 |
| 0 | 1,7 | 1,3 | 1,6 |

Calibri 11

Cortar
Copiar
Pegar
Pegado especial...
Insertar...
Eliminar...
Borrar contenido
Filtrar
Ordenar
Insertar comentario
Formato de celdas...
Elegir de la lista desplegable...
Asignar nombre a un rango...
Hipervínculo...

**Nombre nuevo**

Nombre: UProducidas
Ámbito: Libro
Comentario:
Hace referencia a: =Hoja1!$E$8:$H$8

Aceptar Cancelar

**IMPORTACIÓN Y EXPORTACIÓN DE DATOS A HOJA DE CALCULO CON LINGO:**

Fichero disponible en la dirección **C :\Lingo\EjemploProduccion.xls**:

**DATA:**

!Se incorporan los datos al modelo… ;

Maquina=@OLE('**C:\Lingo\EjemploProduccion.xls** ');

HrsDisponibles=@OLE('**C:\Lingo\EjemploProduccion.xls** ');

Producto=@OLE('**C:\Lingo\EjemploProduccion.xls** ');

Beneficio=@OLE('**C:\Lingo\EjemploProduccion.xls** ');

HrsProdUtilizadas=@OLE('**C:\Lingo\EjemploProduccion.xls**');

!Se guarda la solución en EXCEL;

@OLE('**C:\Lingo\EjemploProduccion.xls**')=UProducidas;

**ENDDATA**

## IMPORTACIÓN Y EXPORTACIÓN DE DATOS A HOJA DE CALCULO CON LINGO:

Por supuesto la solución tras resolver el modelo que toma los datos desde Excel es la misma que se obtuvo inicialmente:

Solution Report - EJEMPLO_PRODUCCION

```
Transfer Method:          OLE BASED
Workbook:                 C:\MisDocumentos\EjemploProduccion1.xlsx
Ranges Specified:                  1
    UPRODUCIDAS
Ranges Found:                      1
Range Size Mismatches:             0
Values Transferred:                4
```

| Variable | Value | Reduced Cost |
|---|---|---|
| HRSDISPONIBLES( ALISADORA) | 28.00000 | 0.000000 |
| HRSDISPONIBLES( CORTADORA) | 34.00000 | 0.000000 |
| HRSDISPONIBLES( SOLDADORA) | 21.00000 | 0.000000 |
| BENEFICIO( P1) | 26.00000 | 0.000000 |
| BENEFICIO( P2) | 35.00000 | 0.000000 |
| BENEFICIO( P3) | 25.00000 | 0.000000 |
| BENEFICIO( P4) | 37.00000 | 0.000000 |
| UPRODUCIDAS( P1) | 0.000000 | 3.805195 |
| UPRODUCIDAS( P2) | 10.00000 | 0.000000 |
| UPRODUCIDAS( P3) | 5.000000 | 0.000000 |
| UPRODUCIDAS( P4) | 0.000000 | 1.441558 |
| HRSPRODUTILIZADAS( ALISADORA, | 1.700000 | 0.000000 |
| HRSPRODUTILIZADAS( ALISADORA, | 2.100000 | 0.000000 |
| HRSPRODUTILIZADAS( ALISADORA, | 1.400000 | 0.000000 |
| HRSPRODUTILIZADAS( ALISADORA, | 2.400000 | 0.000000 |
| HRSPRODUTILIZADAS( CORTADORA, | 1.200000 | 0.000000 |
| HRSPRODUTILIZADAS( CORTADORA, | 2.500000 | 0.000000 |
| HRSPRODUTILIZADAS( CORTADORA, | 1.700000 | 0.000000 |
| HRSPRODUTILIZADAS( CORTADORA, | 2.600000 | 0.000000 |
| HRSPRODUTILIZADAS( SOLDADORA, | 1.700000 | 0.000000 |
| HRSPRODUTILIZADAS( SOLDADORA, | 1.300000 | 0.000000 |
| HRSPRODUTILIZADAS( SOLDADORA, | 1.600000 | 0.000000 |
| HRSPRODUTILIZADAS( SOLDADORA, | 0.8000000 | 0.000000 |

## IMPORTACIÓN Y EXPORTACIÓN DE DATOS A HOJA DE CALCULO CON LINGO:

Que a su vez transfiere los valores óptimos de las variables a Excel:

| | A | B | C | D | E | F | G | H | I |
|---|---|---|---|---|---|---|---|---|---|
| 1 | | | | | | | | | |
| 2 | | | | | | | | | |
| 3 | **EJEMPLO PLANIFICACION DE LA PRODUCCION** | | | | | | | | |
| 4 | | | | | | | | | |
| 5 | | | | | | | | | |
| 6 | BENEFICIO | | | 475 | P1 | P2 | P3 | P4 | |
| 7 | | Beneficio/unidad: | | | **26** | **35** | **25** | **37** | |
| 8 | | Unidades a Producir: | | | 0 | 10 | 5 | 0 | |
| 9 | | | | | | | | | |
| 10 | | | | | | | | | |
| 11 | MAQUINA | | HrsDisponibles | | Hrs. Utilizadas | Hrs. Producción utilizadas por unidad de producto | | | |
| 12 | ALISADORA | | **28** | >= | 28 | **1,7** | **2,1** | **1,4** | **2,4** |
| 13 | CORTADORA | | **34** | >= | 33,5 | **1,2** | **2,5** | **1,7** | **2,6** |
| 14 | SOLDADORA | | **21** | >= | 21 | **1,7** | **1,3** | **1,6** | **0,8** |
| 15 | | | | | | | | | |